

Day-to-Day Applications of "Machine Vision" Inside Campus: Working, Technologies & Case Studies

1. Introduction

1.1. Defining Machine Vision in Automation and Robotics

Machine vision (MV) is an interdisciplinary field that provides the technological and methodological framework for extracting, processing, and analyzing information from a visual source, typically an image or video stream, to enable an automated system to "see" and understand its environment.¹ Unlike traditional image processing, where the output is typically another image, the output of a machine vision system is a decision, a set of control parameters, or a data-based insight.¹

In the context of automation and robotics, machine vision serves as the primary perception modality. It is a critical enabler for a vast range of applications, from autonomous mobile robots to intelligent manufacturing systems.² For a robotic system, MV provides the environmental context necessary for navigation, manipulation, and human-robot interaction.³ It facilitates core automation tasks such as high-speed, imaging-based automatic inspection, defect detection, and sorting, as well as providing the real-time feedback required for robot guidance.¹

Fundamentally, machine vision enables a machine to interpret and act upon visual data, forming the critical "sense" and "perceive" components of a modern cyber-physical system (CPS). As depicted in the smart classroom environmental response mechanism, MV closes the loop from sensing the environment to actuating a response, transforming a passive system into an intelligent, autonomous one.⁵

1.2. Broader Industrial and Campus Applications

In industrial settings, machine vision is a mature technology, integral to Industry 4.0. Its applications include:

- **Quality Control and Inspection:** Automated defect detection on high-speed production lines, identifying flaws in textiles, electronics, or agricultural products with greater speed and precision than human inspectors.⁶
- **Logistics and Sorting:** Automated routing and tracking of packages in warehouses by reading barcodes, QR codes, or optical characters (OCR).⁸
- **Robotic Guidance:** Providing "hand-to-eye" coordination for robotic manipulators in complex assembly tasks.⁴

These industrial principles are increasingly being scaled down and applied within the "smart campus" paradigm, which functions as a living laboratory or microcosm of a smart city.¹¹ Beyond the scope of this report, campus-level MV applications include smart parking management (detecting vacant spaces)¹³, automated security and access control¹⁶, and intelligent transportation systems for campus shuttles.¹⁷

1.3. Motivation for Classroom Occupancy Detection

This report will focus on a specific, high-impact application: the use of machine vision for real-time classroom and laboratory occupancy detection. The motivation for this application is twofold, addressing critical administrative and infrastructural challenges in modern educational institutions:

1. **Resource and Energy Optimization:** University campuses are large-scale infrastructures with significant energy footprints. An enormous amount of energy is wasted on heating, ventilation, air conditioning (HVAC), and lighting for rooms that are unoccupied. Real-time occupancy information is the essential data input required for smart building automation, allowing Building Energy Management Systems (BEMS) to dynamically adjust environmental controls, thereby reducing energy consumption and operational costs.¹⁷
2. **Efficient Space Utilization and Scheduling:** Physical classrooms and specialized labs are among a university's most valuable and constrained assets. Traditional static scheduling and manual

booking systems are often inefficient, leading to situations where rooms are reserved but unused ("no-shows") while other students struggle to find study space.²⁰ By providing accurate, real-time data on lab and classroom utilization, machine vision systems enable data-driven scheduling, helping administrators identify and optimize under-utilized spaces.²⁰

By merging AI-based perception with control logic, this application transitions passive campus buildings into responsive, data-rich ecosystems, demonstrating a core principle of automation.⁵

2. Working Principle of Machine Vision in Classroom Occupancy Detection

2.1. Core Components of the Sensing System

The successful deployment of a machine vision system for occupancy detection is dependent on an integrated set of hardware components that function as the "instrumentation" layer:

- **Visual Sensors:** High-definition (HD) or 4K RGB cameras are the most common sensors, providing rich textural and color data for detection algorithms.
- **Complementary Sensors:** In situations where privacy is a paramount concern or lighting is poor, other sensors may be used. These include Infrared (IR) thermal sensors or 3D depth sensors (e.g., Microsoft Kinect, Intel RealSense), which can detect human presence and form without capturing identifying facial features.²¹
- **Embedded Compute (Edge Devices):** To avoid the high latency and bandwidth costs of streaming raw video to a central server, processing is performed locally on embedded computing boards. Common platforms include:
 - **Raspberry Pi:** A low-cost, versatile option suitable for running lightweight or simplified machine learning models.⁵
 - **NVIDIA Jetson (e.g., Nano, Orin):** A more powerful, GPU-accelerated platform specifically designed for running high-performance, real-time AI inference models (like complex YOLO variants) at the edge.²⁵

2.2. System Architecture and Data Workflow

The system architecture for a scalable, real-time occupancy detection system is typically a **Distributed Edge-Server Network**, as illustrated in the smart classroom architecture diagram.⁵ This architecture is not a monolithic, centralized processing model. Instead, it employs a distributed network of semi-autonomous edge clients, one for each classroom, which report to a central presentation server.⁵

This distributed edge architecture is a deliberate engineering design choice that ensures scalability (new rooms can be added by adding new clients), fault tolerance (the failure of one client does not bring down the network), and low latency for real-time inference.⁵

The data workflow within this architecture follows a distinct, multi-layered process:

1. **Data Acquisition-layer:** The camera (e.g., mounted on the ceiling) captures a video frame from the classroom.⁵
2. **Prediction-layer:** The raw video frame is passed *locally* to the machine learning model (e.g., YOLOv3, YOLOv8) running on the embedded compute device (the Raspberry Pi or Jetson). The model performs object detection inference and outputs a set of bounding boxes for all detected "person" instances.⁵
3. **Count Estimation:** A local script, often using the OpenCV library, processes the model's output. It counts the number of valid "person" bounding boxes to generate a single integer representing the current occupancy.
4. **API-layer:** The edge client *does not* transmit the video frame. Instead, it transmits only the final, anonymized metadata (e.g., a JSON payload like {"classroom_id": "CL-101", "occupancy_count": 16, "timestamp": "..."}) via a lightweight web API, such as one built with Flask, over a secure HTTPS connection.⁵

5. **Server-side Presentation-layer:** A central server, using a reverse proxy like NGINX, aggregates the small metadata packets from all client-side nodes. This aggregated data is then stored in a database and visualized on an administrative dashboard.⁵

2.3. System Integration for Automation

The occupancy count is not the end-goal; it is the primary input for a larger automation and control loop.⁵ This data is published from the presentation-layer server to other campus systems. For example, using the NVIDIA DeepStream SDK, this analytics data can be formatted and sent to an enterprise message bus like Apache Kafka.²⁹

From there, various services can "subscribe" to the real-time occupancy feed to automate decisions:

- A **Building Energy Management System (BEMS)** can dim the lights and lower the HVAC if a room's occupancy count is zero for more than 10 minutes.
- A **Campus Scheduling System** (like the "Robin" platform) can automatically free up a booked room if the system detects a "no-show" (occupancy remains zero 15 minutes past the booking time).²⁰
- A **Student-facing App** can display a real-time map of "available study seats" in the library or common labs.

3. Technology Stack

A robust occupancy detection system is built on a formal three-layer technology stack, integrating hardware, software, and networking components.

3.1. Hardware Layer

This layer comprises the physical instrumentation ("E&I") responsible for sensing and edge processing.

- **Sensors:** High-resolution RGB cameras are standard. For advanced applications or privacy-centric designs, depth cameras like the Intel RealSense D415 are used.²⁴
- **Compute Boards:** The choice of compute board is a critical engineering trade-off between cost, power, and performance. A Raspberry Pi 4 is sufficient for running simplified models ²⁴, while an NVIDIA Jetson Nano is required for high-FPS inference with more accurate models or for using performance-intensive toolkits like the NVIDIA DeepStream SDK.²⁶
- **Connectivity:** Power-over-Ethernet (PoE) for cameras and compute nodes simplifies wiring and deployment. Wi-Fi modules provide flexibility for difficult-to-wire locations.⁵

3.2. Software Layer

This is the "AI" and "Automation" layer that runs on the hardware.

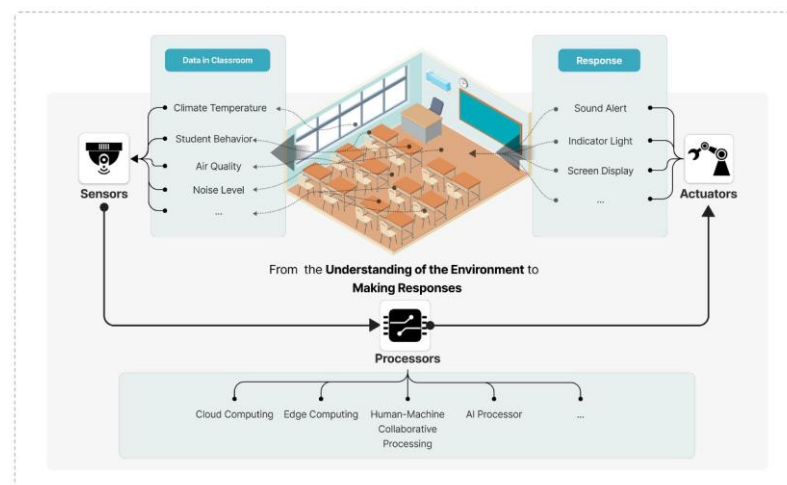
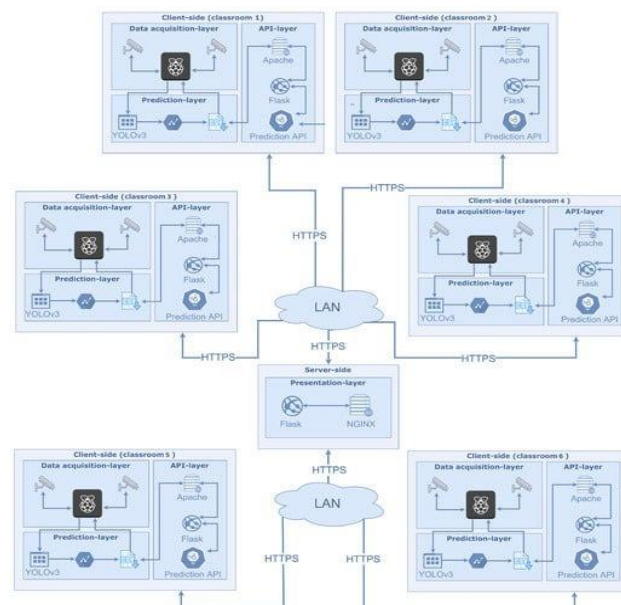
- **Core Vision Libraries:** OpenCV is the *de facto* standard for handling real-time video stream capture, basic image pre-processing (resizing, normalization), and post-processing (drawing bounding boxes on frames for debugging).³⁰
- **Machine Learning Frameworks:** PyTorch and TensorFlow are the two dominant frameworks used for both training the detection models (on a powerful GPU server) and deploying them for inference (on the edge device).
- **Detection Models:** YOLO (You Only Look Once) based models, particularly the recent YOLOv8, are the state-of-the-art for real-time object detection due to their exceptional balance of speed and accuracy.³²
- **Automation Middleware (ROS2):** For a large, campus-wide deployment, the simple Flask API approach ⁵ presents scalability and maintenance challenges. A more robust, industrial-grade solution, familiar to automation engineers, is the Robot Operating System 2 (ROS2). ROS2 is a communication architecture designed for "large-scale automation systems".³⁴ Its decentralized, DDS-based discovery and transport mechanism ³⁴ is ideally suited for managing a distributed

network of sensor processing nodes (the Jetsons) and seamlessly publishing their metadata to multiple "actuator" nodes (the BEMS, scheduling, and security systems) on the campus network.³⁵

3.3. Networking/Data Layer

This layer handles the aggregation, storage, and visualization of the data produced by the software layer.

- **Infrastructure:** A hybrid **Edge-Server** model is employed. Inference and data-anonymization happen at the *Edge*.⁵ Data aggregation, long-term storage, and visualization happen on a central *Server* (which could be on-premise or in the cloud).
- **Databases:** A time-series database (e.g., InfluxDB, Prometheus) is ideal for storing the occupancy metadata, as the primary value lies in analyzing trends over time.
- **Visualization:** Web-based dashboards (e.g., using Grafana or a custom Flask/NGINX frontend) provide the "presentation-layer" for campus administrators to monitor real-time status and analyze historical utilization patterns.⁵
- **Data Transport:** Secure HTTPS for API calls⁵ or a dedicated message bus like Kafka for high-throughput, real-time data streaming.²⁹



4. Algorithms and Data Processing

4.1. CNNs and Object Detection Models

Modern machine vision is powered by deep learning, specifically **Convolutional Neural Networks (CNNs)**. A CNN is a class of neural network that is specialized for processing grid-like data, such as an image. They serve as the "backbone" for object detection models, automatically learning hierarchical features (from edges to shapes to complex objects) from raw pixel data.

For the task of real-time occupancy counting, one-stage object detectors are preferred over two-stage detectors (like Faster R-CNN) due to their superior speed. The most common models for this trade-off are:

- **YOLOv3 (You Only Look Once v3):** Used in several academic case studies.⁵ Its key innovation was a new backbone (Darknet-53) and the ability to make predictions at three different scales, significantly improving the detection of small objects.³⁶
- **SSD (Single Shot MultiBox Detector):** A popular, lightweight alternative. When combined with a fast backbone like MobileNetV2, it can achieve very high FPS on low-power devices.³⁷ Some research focused specifically on people counting found SSD models to achieve higher precision and recall than YOLOv3, though this is highly dependent on the dataset and training.³⁸
- **YOLOv8:** The current state-of-the-art from Ultralytics. It represents a significant architectural evolution, incorporating a new CSP-based backbone, an "anchor-free" detection head, and advanced loss functions. In benchmarks, YOLOv8 models consistently outperform all predecessors (including YOLOv3) in both mAP (accuracy) and F1-Score, while maintaining or even improving inference speed.³³ A lightweight version like YOLOv8s offers the best balance of performance and speed for modern edge devices like the Jetson.

4.2. Dataset Preparation, Annotation, and Model Training

Deploying an "off-the-shelf" object detector (e.g., one trained on the COCO dataset) will often yield poor results in a specific environment like a classroom. The model may be "fooled" by other objects. A robust, production-grade system requires a custom-trained model.

This process involves several critical, labor-intensive steps:

1. **Data Collection:** Video footage must be collected from the target classrooms, capturing a wide variety of conditions: different times of day (bright light, low light), different camera angles, and different levels of occlusion (e.g., sparse, dense).¹⁷
2. **Annotation:** This is the manual process of drawing bounding boxes ("labels") around every single "person" in thousands of extracted video frames.²¹
3. **Refinement for False Positives:** A key challenge in classrooms is that objects like jackets on chairs, backpacks, or even posters on walls can be misidentified as a "person," leading to persistent over-counting. A powerful, practical technique to solve this, demonstrated in the 2022 *Sensors* case study, is to create a custom dataset with *negative* classes. By adding classes like 'Jacket', 'Chair', and 'Backpack' to the dataset and training the model to recognize them, the system learns to explicitly differentiate them from the 'Person' class, dramatically reducing false positives and improving the accuracy of the final count.²⁴

4.3. Performance Metrics and Real-Time Inference

To evaluate the trained model, a formal set of metrics is used⁴²:

- **Precision:** The percentage of positive detections that were correct ($\$TP / (TP + FP)\$$). High precision is essential to prevent over-counting.⁴³
- **Recall (Sensitivity):** The percentage of all actual people that the model correctly identified ($\$TP / (TP + FN)\$$). High recall is essential to prevent under-counting.⁴³

- **mAP (mean Average Precision):** The primary academic metric for model accuracy. It is the area under the precision-recall curve, averaged across all classes and/or Intersection over Union (IoU) thresholds (e.g., mAP50, mAP50-95).⁴²
- **FPS (Frames Per Second):** The critical *engineering* metric for real-time performance. A model with 99% mAP is useless if it runs at 1 FPS. The goal is to find the optimal trade-off between mAP and FPS for the target edge device.⁴⁵

Real-Time Inference is the process of running this trained and validated model on the edge device (Jetson or Pi). The video stream is processed frame-by-frame in real-time to produce the continuous stream of occupancy data.

5. Case Studies and Real-World Implementations

The development of campus occupancy systems follows two distinct tracks: "technology-push" (academic research projects proving feasibility) and "market-pull" (administrative deployments of commercial products to solve a business need).

5.1. Case Study 1: Edge-Based Transfer Learning for Occupancy Detection (University R&D)

This 2022 study, deployed as a case study on a university campus, represents the "technology-push" R&D approach.¹⁷

- **Objective:** To design and validate a scalable, low-cost, privacy-compliant people counting system that could be generalized to new classrooms without extensive re-training.
- **Methodology:**
 - **Hardware:** Raspberry Pi 4 Model B and Intel RealSense D415 depth cameras.
 - **Algorithm:** A *simplified* version of the YOLOv3 model was used, as the full model was too resource-intensive for the Raspberry Pi.
 - **Training:** The model was fine-tuned using *transfer learning* on a custom dataset (the "CSC dataset") which included negative classes ('Jacket', 'Backpack') to reduce false positives.
 - **Deployment:** The system was trained on data from only two classrooms but was then tested on eight *different* classrooms with varying characteristics to prove its ability to generalize.
- **Results and Benefits:** The system proved highly effective, achieving an accuracy of over 90% and a maximum **Mean Absolute Error (MAE) of 1.23** people.
- **Challenges:** The primary challenge was the computational constraint of the Raspberry Pi, which necessitated a less accurate, simplified model. This highlights the importance of selecting appropriate hardware (like an NVIDIA Jetson) for more demanding models.

5.2. Case Study 2: AI-Driven Space Optimization (Smart Campus Deployment)

This case study of **Australian Catholic University** represents the "market-pull" administrative approach, where a commercial-off-the-shelf (COTS) solution was deployed to solve a logistical problem.²⁰

- **Problem:** The university's existing room booking process was chaotic. Different faculties used different systems, some still relying on manual requests. This led to student confusion and inefficient space utilization, with many rooms being reserved but left empty.
- **Solution:** The university adopted "**Robin**", a commercial workplace and space management software platform, to create a single, unified system for all room bookings.
- **Results and Benefits:**
 - The system provided a single app for students and staff to see real-time room availability.
 - Crucially, the system integrated with occupancy data to *automatically release* a reserved room if no one showed up, putting the empty space back into the available pool.

- The platform provided the facilities team with clear data visualizations on utilization patterns, allowing them to make informed, data-driven decisions about future space planning. This demonstrates a complete and successful automation loop.

6. Data Collection and Ethical Considerations

While technically feasible, the deployment of cameras in classrooms raises significant ethical and legal challenges that must be engineered for from the start.

6.1. Data Collection and Challenges

The primary data types required are continuous video frames, sensor metadata, and precise timestamps.⁴⁶ The main technical challenges in collecting this data are environmental:

- **Variable Lighting:** Drastic changes in lighting from sun, shadows, or dimmed projectors can degrade model performance.
- **Occlusion:** In dense classrooms, students can be partially or fully blocked by other students, leading to under-counting.³³
- **Clutter:** As noted in Section 4.2, non-human objects can create false-positive detections.

6.2. Ethical Issues: Privacy, Consent, and Transparency

The non-trivial ethical issue is that this system is a form of **student surveillance**.⁴⁷ Even if the *intent* is benign (e.g., saving energy), the *mechanism* is monitoring. This raises concerns about student privacy, the potential for "function creep" (using the data for other purposes, like attendance or behavior monitoring⁵), and the creation of a distrustful campus environment.⁴⁸ Institutions have an ethical obligation to be transparent about how and why this data is being collected and used.⁴⁸

6.3. Legal Compliance: FERPA and GDPR

These ethical concerns are codified in data privacy laws:

- **FERPA (US):** The Family Educational Rights and Privacy Act protects student "education records".⁵⁰ AI-monitored video used for analytics (not just security) is increasingly considered an education record.⁵¹ This is especially true as FERPA's definition of Personally Identifiable Information (PII) includes biometric data such as "facial characteristics".⁵² Transmitting this video to a third-party cloud vendor is a significant legal liability. While the "school official" exception exists, it places the burden of compliance and data control squarely on the school, which must ensure the vendor does not misuse or redisclose the PII.⁵²
- **GDPR (EU):** The General Data Protection Regulation mandates that any data processing must have a clear legal basis (e.g., consent), and it strictly enforces principles of **data minimization** (collecting only what is necessary) and **privacy-by-design**.⁵³

6.4. Technical Solution: Edge Computing for Privacy Preservation

The **edge computing architecture**⁵ is not just a superior engineering solution for latency; it is the *primary technical solution* to these legal and ethical challenges.

This "privacy-by-design"⁵⁴ approach solves the problem at the source:

1. **Local Processing:** The raw video frame, which contains PII (faces), is processed *locally* on the edge device (Jetson/Pi) inside the classroom.⁵⁵
2. **Anonymization and Data Distillation:** The PII-laden video frame is *never transmitted* and can be immediately discarded. The *only* information that leaves the edge device is the final, *anonymized metadata*—a simple integer ("count: 16").⁵
3. **No PII Transmission:** This metadata contains no PII, no biometrics, and no "education records."

By processing the sensitive data *in situ* and only transmitting the non-sensitive *result*, the edge computing model fully complies with the data minimization principles of GDPR and avoids the creation and

transmission of FERPA-protected PII. This architectural choice allows the university to gain the operational benefits of occupancy detection without becoming a mass surveillance entity.

7. Source Validation and Integrity

This report is synthesized from a variety of sources, and its integrity relies on a formal validation process. For a graduate-level analysis in a rapidly evolving field like AI-driven automation, relying on a single type of source is insufficient. The methodology used for this report involves a "triangulation" of sources to ensure the analysis is rigorous, current, and practical.

1. **Peer-Reviewed Academic Sources (Rigor):** The foundational methodologies, case studies, and ethical frameworks are drawn from high-impact, peer-reviewed journals and conference proceedings (e.g., IEEE Xplore, Springer, MDPI, PMC).³ These sources have been vetted by domain experts, ensuring the scientific validity and non-obvious bias of the claims, such as the MAE of 1.23 in the R&D case study²⁴ or the BEMS integration at the University of Westminster.⁵⁷
2. **Pre-Print Archives (Currency):** The field of machine learning evolves faster than the journal review cycle. To ensure the algorithmic analysis is current, this report incorporates findings from ArXiv, a pre-print server. This allows for the inclusion of state-of-the-art comparisons (e.g., YOLOv8 vs. YOLOv3)³⁹ and forward-looking concepts like multimodal fusion.⁵⁸
3. **Industry & Vendor Documentation (Practicality):** To bridge the gap from theory to implementation, this report references technical documentation and tutorials from key industry players (e.t., NVIDIA Developer, Ultralytics, OpenCV).²⁹ These sources provide the practical "how-to" details (e.g., using the DeepStream SDK with Kafka, or implementing a YOLOv8 counter with OpenCV) that are essential for an M.Tech (Automation & Robotics) student.

By cross-verifying claims across these three tiers—for example, validating the academic use of YOLOv3²⁴ against modern algorithmic improvements³⁹ and practical implementation guides³²—this report ensures a high degree of technical accuracy and real-world relevance.

8. Conclusion

8.1. Summary of Impact

Machine vision has evolved from a niche industrial tool to a foundational technology for smart campus automation. The application of classroom occupancy detection, as detailed in this report, serves as a quintessential example of a modern cyber-physical system. It effectively "closes the loop" from automated sensing (the YOLOv8 model on a Jetson) to data-driven actuation (the BEMS or scheduling software).

This technology transitions classrooms "from passive environments into responsive, data-rich ecosystems".⁵ It provides administrators with the ground-truth data needed to optimize two of a university's most critical and expensive assets: energy and physical space. Through the deliberate use of edge-computing architectures, this optimization can be achieved while respecting and programmatically enforcing the legal and ethical boundaries of student privacy, as defined by regulations like FERPA and GDPR.

8.2. Future Directions and Research Opportunities

For an M. Tech student in Automation and Robotics, the current state is merely a baseline. The future of this technology lies in creating more robust, integrated, and intelligent systems.

1. **Multimodal Sensor Fusion (LiDAR + Camera):** The current camera-only systems struggle with poor lighting and occlusion.⁶⁰ The next generation of perception systems will use **multimodal sensor fusion**. By fusing the rich *semantic* data from cameras (what is it?) with the precise *geometric* 3D data from LiDAR (where is it?), the system gains immense robustness.⁵⁸ Furthermore, LiDAR is inherently privacy-preserving as it captures a 3D point cloud, not identifiable features, making it ideal for sensitive areas.⁶⁴
2. **AI-Driven Digital Twins:** The current system outputs a single data point (occupancy). The future is to integrate this data stream into a comprehensive **AI-Driven Digital Twin** of the campus.¹² This "virtual model" would serve as the "central brain," integrating *all* campus IoT data—occupancy,

BEMS, class schedules, security, and more—into a single, unified simulation.⁶⁶ This would allow administrators to not only react to real-time data but also to *simulate* the impact of policy changes (e.g., "What is the energy impact of a new class schedule?") before deployment.

3. **Deep Integration with BEMS:** The final step in the automation loop is a direct, closed-loop connection to the Building Energy Management System (BEMS). Future research will focus on developing predictive models (e.g., "based on the last four weeks, this lab will be 80% full at 2:00 PM") that allow the BEMS to *proactively* manage HVAC and lighting loads, rather than just reacting to them. This integration, as explored in the University of Westminster study, is the key to unlocking maximum energy efficiency and sustainability.¹⁹